

Programação em C++



Prof. Daniel Santos

daniel.sampaio@sp.senai.br

Senai Roberto Simonsen

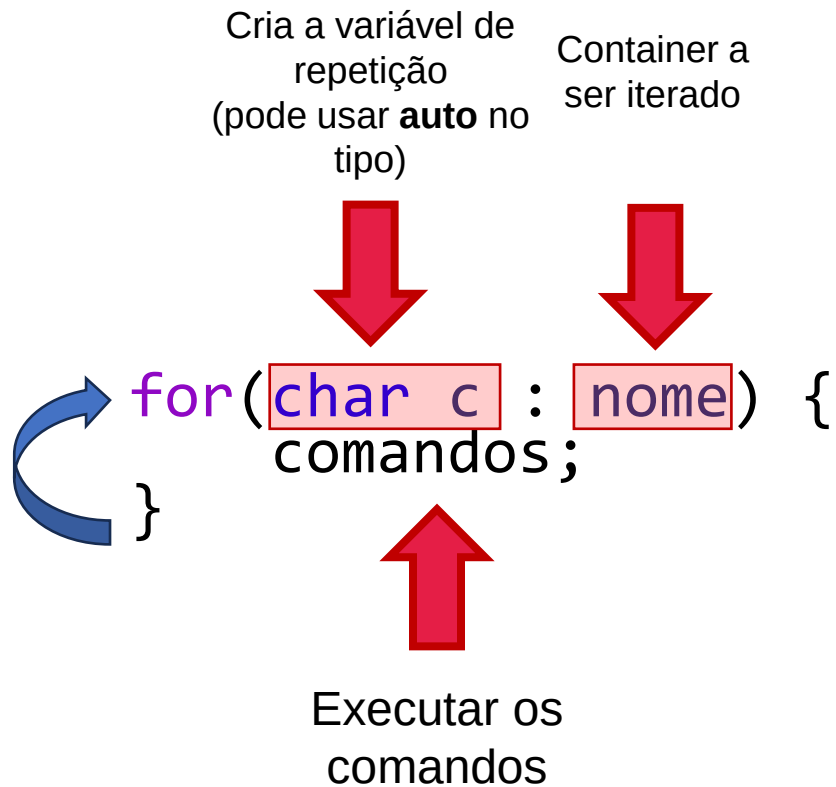
R. Monsenhor Andrade, 298 - Brás, São Paulo - SP



Prof. Daniel Santos

For-each (ranged based for)

O **for-each** é uma forma simplificada e moderna de percorrer todos os elementos de um container (como array, vector, string, map) sem precisar usar índice.



A variável de repetição terá o valor de cada elemento do container a cada ciclo

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string nome = "Daniel";

    for (auto c : nome) {
        cout << c << endl;
    }
}
```

Container

Em programação, muitas vezes precisamos armazenar vários valores juntos, como uma lista de nomes, números ou itens.

Em vez de criar uma variável para cada valor, usamos **containers**, que são estruturas que nos permitem agrupar dados de forma organizada e eficiente.

- **Array**: Ordenado, Repetível, Acesso por Índice, Tamanho Fixo;
- **Vector**: Ordenado, Repetível, Acesso por Índice, Tamanho Dinâmico;
- **Set**: Ordenado, Sem Repetição, Sem acesso por Índice, Tamanho Dinâmico;
- **Map**: Ordenado, Sem Repetição, Acesso por Chave, Tamanho Dinâmico.

OBS: string pode ser considerado um **pseudo-container**, pois tem características similares.

Containers



Vantagens de cada um dos principais containers:

Estrutura	Ordenado	Itens Duplicados	Acesso aos elementos	Tamanho Dinâmico	Vantagens
array	Sim	Sim	Por índice: ultra rápido Por item: lento	Não	Baixo uso de memória e CPU
vector	Sim	Sim	Por índice: ultra rápido Por item: lento (se não estiver ordenado) rápido (se estiver ordenado)	Sim	Flexível
set	Automaticamente	Não	Por item: rápido	Sim	Ordenado e Busca Rápida
map	Automaticamente	Chaves: Não Valores: Sim	Por chave: rápido	Sim	Associa chaves a valores, ordenado

<https://en.cppreference.com/w/cpp/container>

Array – array (tradicional)

- Array é uma estrutura que guarda vários dados do mesmo tipo.
- Os dados são guardados em posições (índices).
- Tamanho fixo: não muda depois de criado.

```
int valores[4]; // Cria array com 4 itens (índices de 0 a 3)

int valores[4] = {10, 8, 5, 7}; // Cria array com valores definidos
int valores[] = {10, 8, 5, 7}; // Cria array com valores definidos e deduz que são 4 itens

valores[2] = 11; // Altera índice [2] de 5 para 11

cout << valores[0]; // Mostra o valor do índice [0] --> 10

// Número de elementos de um array
int tamanho = sizeof(valores) / sizeof(valores[0]);

// Acessar cada item do array
for (int i = 0; i < 4; i++) {
    cout << valores[i] << endl;
}
```

Array – std::array (moderno)

- É uma estrutura fixa como o array tradicional.
- Mas é mais seguro e poderoso, pois oferece **métodos** (como .size()).
- Precisa incluir a biblioteca: #include <array>

```
#include <iostream>
#include <array>
using namespace std;

int main()
{
    array<int, 4> valores; // Cria array com 4 itens (índices de 0 a 3)

    array<int, 4> valores = {10, 8, 5, 7}; // Cria array com valores definidos

    valores[2] = 11; // Altera índice [2] de 5 para 11

    cout << valores[0] << endl; // Mostra valor do índice [0] --> 10

    int tamanho = valores.size(); // Número de elementos do array

    // Permite range-based for para acessar itens diretamente
    for (int valor : valores) {
        cout << valor << endl;
    }
}
```

Array - Exemplos



```
// Jogar 1 dado 1000 vezes
#include <iostream>
#include <cstdlib> // rand()
#include <ctime>    // time()

using namespace std;

int main()
{
    int resultados[6] = {0};

    srand(time(0));

    for (int i = 0; i < 1000; i++) {
        int dado = rand() % 6 + 1;
        resultados[dado-1]++;
    }

    for (int i = 0; i < 6; i++) {
        cout << "Número " << i+1 << ": " << resultados[i] << endl;
    }
}
```

array

```
// Jogar 1 dado 1000 vezes
#include <iostream>
#include <cstdlib> // rand()
#include <ctime>    // time()
#include <array>    // std::array

using namespace std;

int main()
{
    array<int, 6> resultados = {0};

    srand(time(0));

    for (int i = 0; i < 1000; i++) {
        int dado = rand() % 6 + 1;
        resultados[dado - 1]++;
    }

    for (int i = 0; i < resultados.size(); i++) {
        cout << "Número " << i+1 << ": " << resultados[i] << endl;
    }
}
```

std::array

Array - Exemplos



```
// Escolher Personagem
#include <iostream>

using namespace std;

int main()
{
    string personagens[] = {"Guerreiro", "Mago", "Arqueiro", "Paladino", "Ninja"};

    int tamanho = sizeof(personagens) / sizeof(personagens[0]);

    cout << "Escolha seu personagem:\n";
    for (int i = 0; i < tamanho; i++) {
        cout << i + 1 << " - " << personagens[i] << endl;
    }

    int escolha;
    cout << "\nDigite o número do personagem: ";
    cin >> escolha;

    if (escolha >= 1 && escolha <= tamanho) {
        cout << "\nVocê escolheu: " << personagens[escolha - 1] << "!" << endl;
    }
    else {
        cout << "Opção inválida!" << endl;
    }
}
```

Array – std::array – Métodos



Método	Descrição
size()	Retorna o número de elementos do array
at(i)	Retorna o valor no índice i com verificação de limites (gera erro out_of_range se i não for um índice válido)
front()	Retorna o primeiro elemento
back()	Retorna o último elemento
fill(valor)	Preenche todos os elementos com o valor passado
swap(outro array)	Troca o conteúdo com outro array de mesmo tipo e tamanho
begin() / end()	Retorna iteradores para o início e o fim (usado em loops e algoritmos)
rbegin() / rend()	Iteradores reversos para percorrer de trás para frente

Vector

- É uma estrutura que armazena vários dados do mesmo tipo.
- Ao contrário do array, não tem tamanho fixo – pode crescer ou diminuir.
- Requer a inclusão da biblioteca <vector>.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> valores; // Cria um vector vazio

    vector<int> valores = {10, 8, 5, 7}; // Cria um vector com valores iniciais

    valores.push_back(4); // Adiciona valor ao final do vector

    valores[2] = 11; // Altera o índice [2] de 5 para 11

    cout << valores[0] << endl; // Mostra valor no índice [0] --> 10

    int tamanho = valores.size(); // Número de elementos

    // Acessar cada item do vector
    for (int i = 0; i < tamanho; i++) {
        cout << valores[i] << endl;
    }
}
```

Vector – Métodos



Método	Descrição
size()	Retorna o número de elementos do vector
at(i)	Retorna o valor no índice i com verificação de limites (gera erro out_of_range se i não for um índice válido)
front()	Retorna o primeiro elemento
back()	Retorna o último elemento
fill(valor)	Preenche todos os elementos com o valor passado
swap(outro array)	Troca o conteúdo com outro vector de mesmo tipo e tamanho
begin() / end()	Retorna iteradores para o início e o fim (usado em loops e algoritmos)
rbegin() / rend()	Iteradores reversos para percorrer de trás para frente

Vector – Métodos



Método	Descrição
push_back()	Adiciona um elemento no final
pop_back()	Remove o último elemento
insert(pos, val)	Insere elemento na posição desejada
erase(pos)	Remove elemento de uma posição
clear()	Remove todos os elementos
resize(n)	Ajusta o tamanho do vetor para n elementos
reserve(n)	Reserva espaço na memória para n elementos

Vector - Exemplo



```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main()
{
    vector<string> nomes = {
        "Carlos", "Beatriz", "Ana", "Lucas", "Pedro", "Mariana", "João", "Fernanda"
    };

    // Remover "Pedro"
    nomes.erase(nomes.begin() + 4);

    // Adicionar novo nome no final
    nomes.push_back("Gabriel");

    // Mostrar nomes que contêm a letra 'e'
    cout << "Nomes que contêm a letra 'e':\n";
    for (auto nome : nomes) {
        for (char c : nome) {
            if (tolower(c) == 'e') {
                cout << "- " << nome << "\n";
                break;
            }
        }
    }
}
```


Set

- É uma estrutura que armazena valores únicos de um mesmo tipo.
- Mantém os dados ordenados automaticamente e não permite itens repetidos.
- Requer a inclusão da biblioteca <set>.

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> valores; // Cria um set vazio
    set<int> valores = {10, 8, 5, 7}; // Cria um set com valores

    valores.insert(4); // Adiciona mais um valor

    // Não é possível acessar por índice, mas podemos verificar existência
    if (valores.find(5) != valores.end()) {
        valores.erase(5); // Remove o valor 5
        valores.insert(11); // "Substitui" 5 por 11 (não é direto como no vector)
    }

    // Mostrar todos os valores do set
    cout << "Valores no set:\n";
    for (int v : valores) {
        cout << v << endl;
    }
}
```

Set - Exemplo



```
#include <iostream>
#include <set>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    set<int> sorteio;
    srand(time(0));

    while (sorteio.size() < 6)
        sorteio.insert(rand() % 60 + 1);

    cout << "Números sorteados: ";
    for (int num : sorteio)
        cout << num << " ";
    cout << endl;
}
```

Map

- É uma estrutura que armazena pares chave-valor, onde cada chave é única.
- Permite acessar rapidamente um valor associado a uma chave.
- Requer a inclusão da biblioteca <map>.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, string> usuario;

    // Inserir campos e valores
    usuario["nome"] = "Daniel";
    usuario["email"] = "daniel@email.com";
    usuario["profissao"] = "Desenvolvedor";

    usuario["profissao"] = "Analista de Sistemas"; // Atualizando dados

    // Mostrando os dados
    cout << "Dados do usuário:\n";
    for (auto& campo : usuario) {
        cout << campo.first << ": " << campo.second << endl;
    }
}
```

Map - Exemplo



```
#include <iostream>
#include <vector>
#include <map>
#include <string>
using namespace std;

int main() {
    vector<map<string, string>> personagens = {
        {{ "nome", "Guerreiro"}, {"vida", "100"}, {"ataque", "30"}, {"defesa", "20"}},
        {{ "nome", "Mago"}, {"vida", "80"}, {"ataque", "40"}, {"defesa", "15"}},
    };

    for (auto personagem : personagens) {
        string nome = personagem["nome"];
        int vida = stoi(personagem["vida"]); // conversão string -> int
        int ataque = stoi(personagem["ataque"]);
        int defesa = stoi(personagem["defesa"]);

        cout << "Personagem: " << nome << endl;
        cout << "  Vida: " << vida << endl;
        cout << "  Ataque: " << personagem["ataque"] << endl;
        cout << "  Defesa: " << personagem["defesa"] << "\n" << endl;
    }
}
```